

Personal Leadership Framework

A working document for the road ahead

Introduction

Until December 2025 I was a software engineer. That was my whole frame of reference. Leadership, change management, organizational dynamics those were things happening somewhere above me, decisions that arrived in my inbox as Jira tickets or Slack announcements. I paid attention to the code, to the architecture, to whether my PR got through review cleanly. I didn't spend much time thinking about the people side of building technology, and honestly, I didn't think I needed to.

When I started this course, it was partly because I'm in the middle of a career transition moving from software engineering into data engineering with an AI specialization and I figured a broader perspective wouldn't hurt while I was job hunting. What I didn't expect was how much of my existing mental model the course would quietly rearrange. I came in thinking leadership was something you grew into once you got a title. I'm leaving with the uncomfortable realization that the people I've respected most in my career were already doing it, years before anyone made them a manager.

In my initial self-assessment I rated my change management skills low maybe a three because I'd never led a change effort in any formal sense. My leadership score wasn't much higher. What I did rate more generously were things like technical communication, giving and receiving code review feedback, and working across time zones. Looking back, I think I was right to be humble about the formal categories and a little too generous on the technical ones. Writing a clear design doc is not the same as helping a team navigate a disagreement, and I'd been conflating the two.

This document is where I put down what I've picked up, what I still have to work on, and the kind of AI/ML leader I want to grow into, starting from exactly where I am right now mid-transition, actively interviewing, and a lot earlier on this road than I used to think I was.

Reflections on Growth and Learning

A few specific moments from the course stuck with me, and I want to write them down before they fade.

The first was the module on change management frameworks. I'd always assumed those were corporate wallpaper the kind of slides executives show on all-hands calls. But working through them carefully, I recognized every stage in rollouts I'd lived through as an engineer, and I saw why some of those rollouts felt smooth and others felt like being dragged. The difference wasn't the technology. It was whether anyone took the early stages seriously the part where you help people understand why the change matters before you hand them the new tool. I'd been on the receiving end of both and hadn't known what I was looking at.

The second was a discussion around a team that deployed an ML model that hit its accuracy targets but failed badly on a subgroup nobody had thought to check. The lesson wasn't about the math. It was about who was, and wasn't, in the room when the validation plan got written. As a software engineer I'd thought of bias and fairness as a testing problem something you caught at the end. Now I see it as a staffing problem, a design problem, a leadership problem. The model is downstream of every earlier decision.

The third moment was smaller but personal. One of the readings talked about influence without authority how the strongest contributors in technical organizations aren't necessarily the ones with the biggest titles, but the ones people trust enough to bring problems to early. I realized I'd been treating my software engineering career as a purely technical game, optimizing for skill and output while underinvesting in the human layer. That's a leadership failure even for an individual contributor. I just hadn't called it that.

My perspective on AI and ML has shifted in two ways that feel important. I used to see ethics as a compliance function a checklist at the end of the project. I now think the ethical questions are the work itself, woven into decisions about what data to collect, what to build, who to build it with, and what to measure. The second shift is about the role I want to play. I came in assuming my job would always be to build the systems while someone else handled the human side. That split doesn't hold. If I want to do this work well especially in AI, where the human consequences compound quickly I have to be fluent in both.

Vision as an AI/ML Leader

I'm not a leader yet in any formal sense, and I want to be honest about that. My next job is a data engineering role, not a management position. But the course convinced me that leadership is a practice you start before you hold the title, or you're not ready when the title arrives. So I want to use the early years of this next chapter to build that practice deliberately.

The kind of leader I want to become is someone who keeps the technical depth. I've seen engineers get promoted away from the work and slowly lose the ability to evaluate what their teams are actually doing, and I don't want to become that person. I want to be the leader who can still read a pipeline, still reason about a model's failure modes, still push back with specifics when something doesn't add up. At the same time, I want to be the one who sees the people. Who knows when a teammate is burning out, who makes space for the quietest voice in the review, who doesn't confuse being hard to please with being rigorous.

The impact I hope to have on whatever organization I join is small at first and grows over time. I'd like to be the new hire people are glad they brought on not because I'm the loudest in the room, but because the work around me gets a little more careful when I'm part of it. Longer term, I want to contribute to an AI and data engineering culture that takes responsibility seriously, where shipping fast and shipping well aren't treated as opposites.

Beyond the organization, I'd like to be a small positive example in the broader community of engineers moving into AI work. A lot of us are doing this transition right now, and the profession is figuring itself out in real time. I can be one more person in the room pulling in a good direction.

Mission Statement

To grow into a data and AI engineering leader who pairs real technical depth with real care for the people affected by the systems I help build starting now, in the everyday choices I make as an engineer, long before any title catches up.

Core Values

I landed on four values, on purpose. Long lists look impressive, but you can't actually hold yourself accountable to twelve things at once. These are the ones I want to use when the choice is harder than it looks.

Craft over title. The engineers I've respected most were already leading before anyone made it official. They wrote clearer docs, asked better questions in review, noticed when a teammate was stuck. I want to treat leadership the same way I've treated engineering as a craft to practice daily, not a promotion to wait for. Whatever role I land next, the behaviors start on day one.

Honesty about what I don't know. I'm new to data engineering at a professional level, new to AI in production, new to leadership of any kind. Pretending otherwise would be worse than useless. I commit to being direct about my limits with interviewers, with future teammates, with myself and to treating gaps as things to close rather than things to hide. Especially in AI work, where confident-sounding wrongness is the most expensive kind.

Respect for the person on the other end of the model. Every data pipeline I help build and every model it feeds ends up affecting someone. A customer whose experience gets shaped by a recommendation. A worker whose task gets automated. A person whose data we never asked permission to use in quite the way we ended up using it. My faith teaches me that each of those people carries inherent worth, and that conviction has to show up in the technical choices, not just in company values decks. Decisions about data and AI need to be made with the affected person in mind, even when they're not in the room.

Curiosity over defensiveness. The instinct I want to build is to treat pushback as information rather than a threat. When someone challenges a pipeline design, a model choice, or a decision I've made, the right first response is not to defend — it's to ask what they're seeing that I'm not. This is harder than it sounds, especially during a career transition when I'm already feeling like I have something to prove.

Key Objectives

Five objectives for the next twelve months. Some are about landing the right role. Some are about becoming the kind of engineer who's ready to lead when the time comes. Each one has a way to measure it, because a goal without a way to measure it is just a wish.

1. Land a data engineering role with AI exposure by the end of Q2. Concrete measures: at least one offer in hand, with a role that includes either model-adjacent work (feature pipelines, ML platform, retrieval systems) or a clear near-term path to it. I'm tracking applications, callbacks, and interview-to-offer ratios weekly so I can tell if my approach is working or if I need to change something.
2. Build a portfolio project that demonstrates both engineering and judgment, not just code. By the end of Q2 I want a deployed end-to-end data pipeline feeding a small ML use case, with a written README that explains the design trade-offs, failure modes, and what I'd do differently. The writing matters as much as the code. Anyone can push to GitHub.
3. In the first ninety days of the new role, practice leadership behaviors that don't require a title. Ask more clarifying questions in meetings than I answer. Offer to write the design doc nobody wants to write. Give teammates specific, useful code review feedback. Measure success by asking my manager at the ninety-day mark for honest feedback on whether those behaviors showed up.
4. Find a mentor who is five to seven years ahead of me on this path and meet with them at least once a quarter. I've historically been allergic to mentorship because I don't like looking like I don't know things. That's exactly the instinct this course taught me to push back against.
5. Write and publish one technical piece this year on something I actually learned the hard way — a blog post, a talk, a detailed write-up of a project. Writing forces clarity, and I don't trust my own understanding until I've had to explain it to someone who can push back.

Action Plans

Short term, the next ninety days. Maintain a consistent job search cadence: targeted applications each week, outreach to at least three people in the field each week, and honest tracking of what's working instead of just grinding through volume. Finish the portfolio project and put it in a state I'd be proud to show a hiring manager. Pick one technical area to deepen for me, probably orchestration and ML pipeline patterns and actually finish the book or course, not just collect tabs. Reach out to two or three people I'd genuinely want as a mentor and ask directly, not with vague “let's grab coffee sometime” language.

Medium term, three to twelve months. Once I'm in the role, spend the first quarter mostly listening. Understand the data, the tooling, the norms, and the people before I try to change anything. Then start

finding small, visible ways to contribute beyond just my tickets improving documentation, mentoring an even newer engineer, taking ownership of a reliability problem nobody wants. Start drafting the technical write-up I committed to. Keep going to the mentor meetings even when I don't feel like I have anything interesting to bring.

Long term, one to three years. Grow into a senior engineer or tech lead role where I'm expected to own outcomes, not just outputs. Mentor at least one person coming up behind me someone making the same kind of transition I'm making now. Build a small but real public presence around data and AI engineering done thoughtfully. Not for vanity, but because credibility comes partly from being willing to put your thinking where people can push back on it.

Evaluation and Adaptation

I've seen enough personal development documents die in folders to know that without a review schedule, this one becomes decoration. So here's how I'll keep it honest:

Every Sunday evening, fifteen minutes. A short journal entry against the values and objectives. Not an exhaustive audit just the one useful question: did I live these values this week, or did I cut corners when nobody was watching? During the job search specifically, the same review is where I check whether my application and networking cadence is actually what I committed to, or whether I've been coasting.

Every quarter, a longer ninety-minute review. I'll write short answers to three questions: What's working? What's not? What do I need to change? Once I have a mentor, this is what I'll bring to them.

Every six months, I'll ask for honest feedback from people in a position to see me clearly a peer I trust, a former colleague, my manager once I have one. I'm specifically going to ask whether my stated values match my behavior, and I want the gap, not the flattery. If nobody can point to anything, I should assume they don't trust me enough yet to be candid, and that's its own problem to work on.

Every year, I'll rewrite this document from scratch. Not revise rewrite. If my understanding of what this work requires hasn't evolved enough to change the language, I haven't been paying attention.

This framework is a draft, not a monument. I fully expect parts of it to look naive a year from now, once I'm actually in the role and learning things I can't anticipate from here. That's the point. I'd rather have something imperfect and real than something polished and hollow, and I'd rather start the practice early and badly than wait until I feel ready.